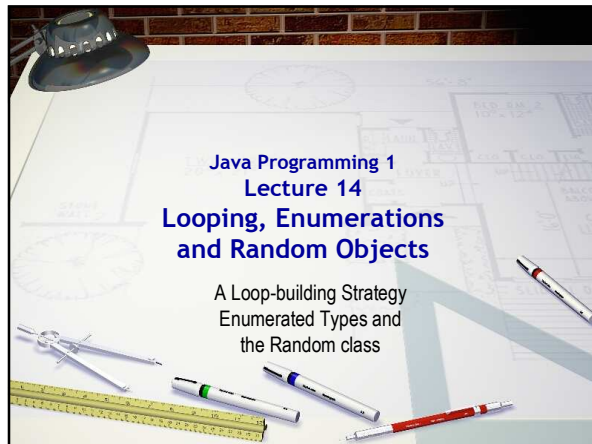
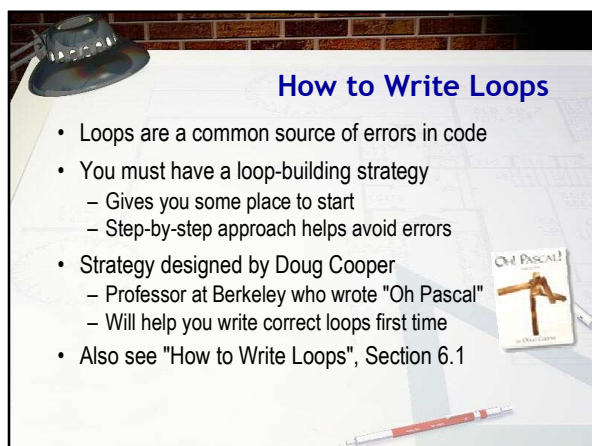
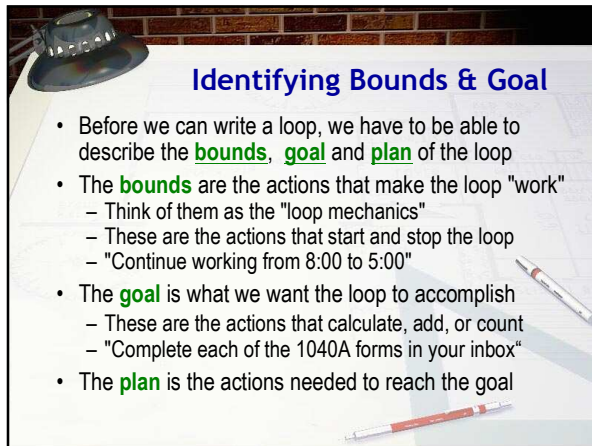




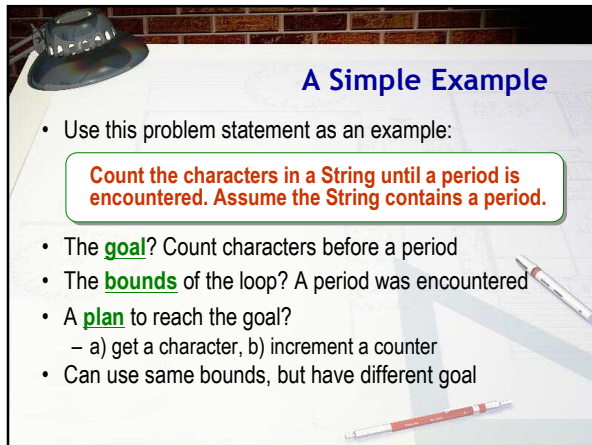






Identifying Bounds & Goal

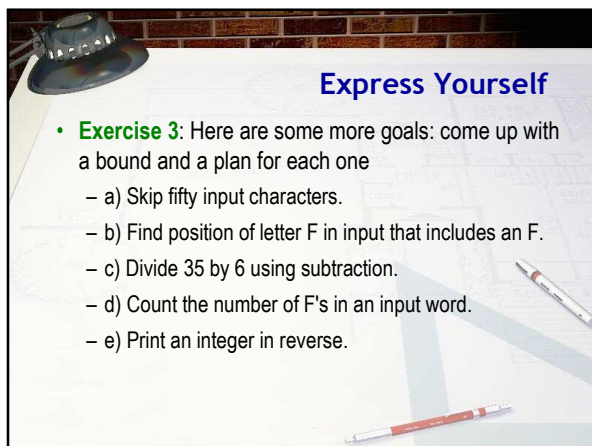
- Before we can write a loop, we have to be able to describe the **bounds**, **goal** and **plan** of the loop
- The **bounds** are the actions that make the loop "work"
 - Think of them as the "loop mechanics"
 - These are the actions that start and stop the loop
 - "Continue working from 8:00 to 5:00"
- The **goal** is what we want the loop to accomplish
 - These are the actions that calculate, add, or count
 - "Complete each of the 1040A forms in your inbox"
- The **plan** is the actions needed to reach the goal



A Simple Example

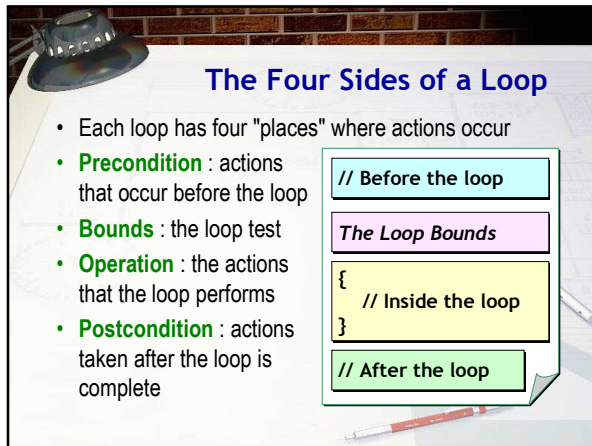
- Use this problem statement as an example:

Count the characters in a String until a period is encountered. Assume the String contains a period.
- The **goal**? Count characters before a period
- The **bounds** of the loop? A period was encountered
- A **plan** to reach the goal?
 - a) get a character, b) increment a counter
- Can use same bounds, but have different goal



Express Yourself

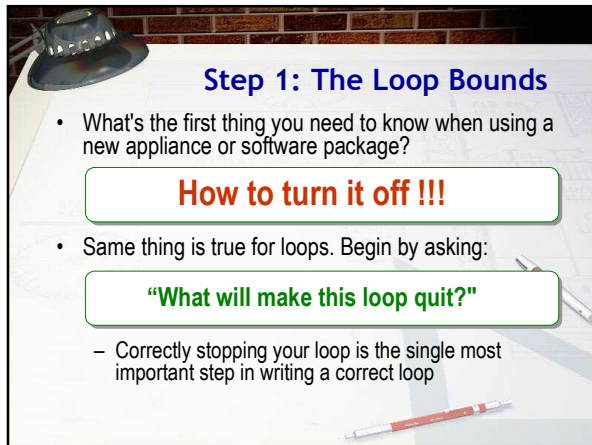
- **Exercise 3:** Here are some more goals: come up with a bound and a plan for each one
 - a) Skip fifty input characters.
 - b) Find position of letter F in input that includes an F.
 - c) Divide 35 by 6 using subtraction.
 - d) Count the number of F's in an input word.
 - e) Print an integer in reverse.



The Four Sides of a Loop

- Each loop has four "places" where actions occur
- Precondition** : actions that occur before the loop
- Bounds** : the loop test
- Operation** : the actions that the loop performs
- Postcondition** : actions taken after the loop is complete

```
// Before the loop
The Loop Bounds
{
    // Inside the loop
}
// After the loop
```



Step 1: The Loop Bounds

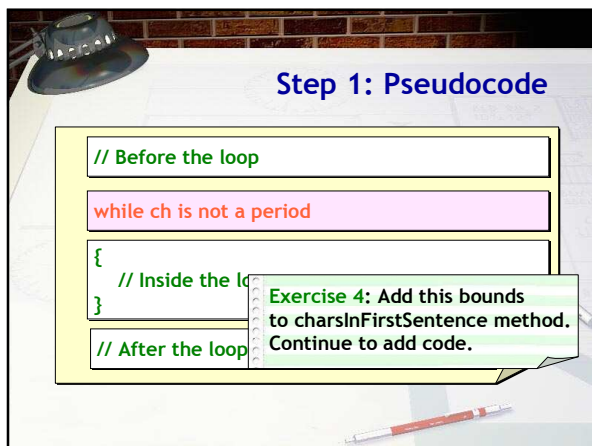
- What's the first thing you need to know when using a new appliance or software package?

How to turn it off !!!

- Same thing is true for loops. Begin by asking:

"What will make this loop quit?"

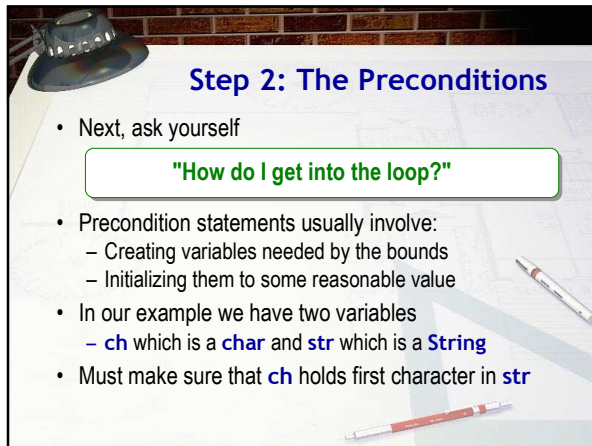
- Correctly stopping your loop is the single most important step in writing a correct loop



Step 1: Pseudocode

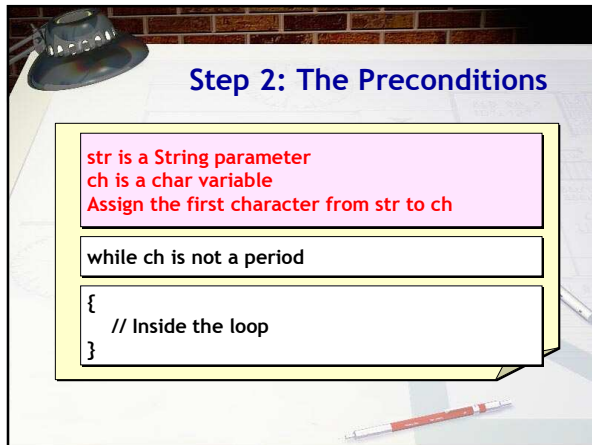
```
// Before the loop
while ch is not a period
{
    // Inside the loop
}
// After the loop
```

Exercise 4: Add this bounds to `charsInFirstSentence` method. Continue to add code.



Step 2: The Preconditions

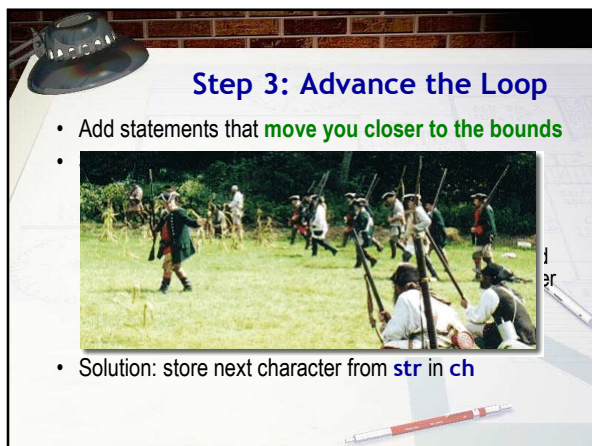
- Next, ask yourself
"How do I get into the loop?"
- Precondition statements usually involve:
 - Creating variables needed by the bounds
 - Initializing them to some reasonable value
- In our example we have two variables
 - ch** which is a **char** and **str** which is a **String**
- Must make sure that **ch** holds first character in **str**



Step 2: The Preconditions

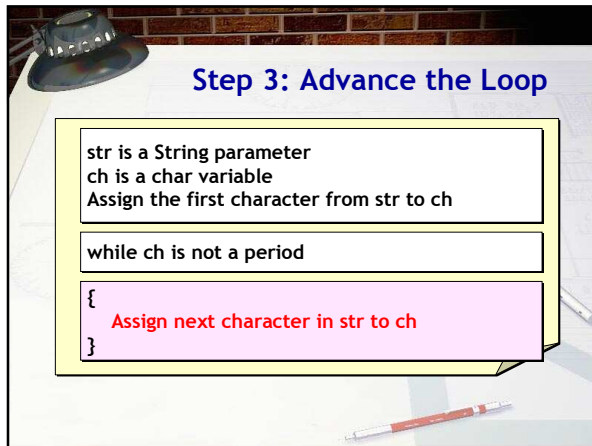
str is a String parameter
ch is a char variable
Assign the first character from str to ch

```
while ch is not a period  
{  
    // Inside the loop  
}
```



Step 3: Advance the Loop

- Add statements that **move you closer to the bounds**
- 
- Solution: store next character from **str** in **ch**

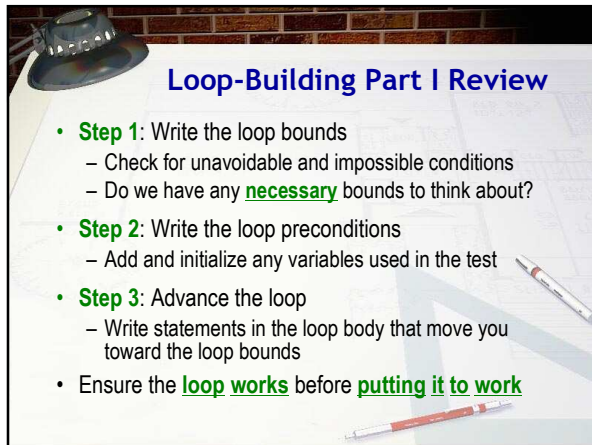


Step 3: Advance the Loop

```
str is a String parameter  
ch is a char variable  
Assign the first character from str to ch
```

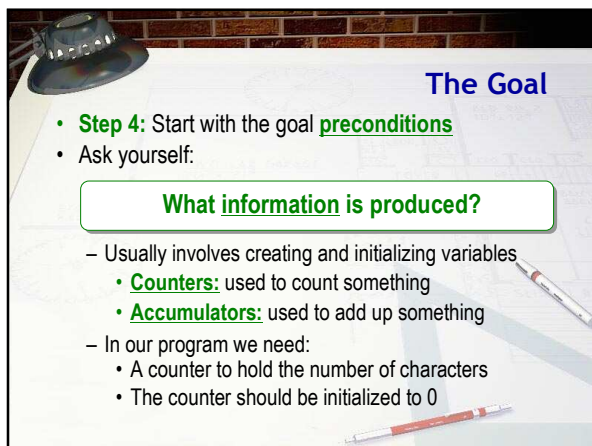
```
while ch is not a period
```

```
{  
    Assign next character in str to ch  
}
```



Loop-Building Part I Review

- **Step 1:** Write the loop bounds
 - Check for unavoidable and impossible conditions
 - Do we have any **necessary** bounds to think about?
- **Step 2:** Write the loop preconditions
 - Add and initialize any variables used in the test
- **Step 3:** Advance the loop
 - Write statements in the loop body that move you toward the loop bounds
- Ensure the **loop works** before **putting it to work**

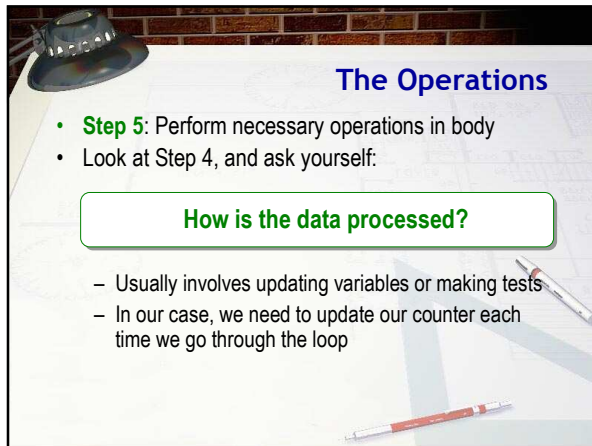


The Goal

- **Step 4:** Start with the goal **preconditions**
- Ask yourself:

What information is produced?

 - Usually involves creating and initializing variables
 - **Counters:** used to count something
 - **Accumulators:** used to add up something
 - In our program we need:
 - A counter to hold the number of characters
 - The counter should be initialized to 0

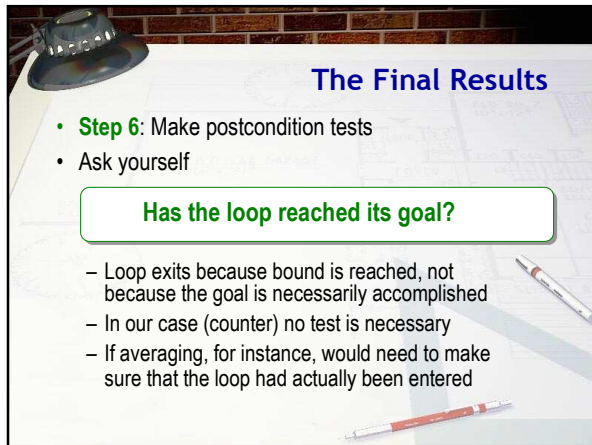


The Operations

- **Step 5:** Perform necessary operations in body
- Look at Step 4, and ask yourself:

How is the data processed?

- Usually involves updating variables or making tests
- In our case, we need to update our counter each time we go through the loop

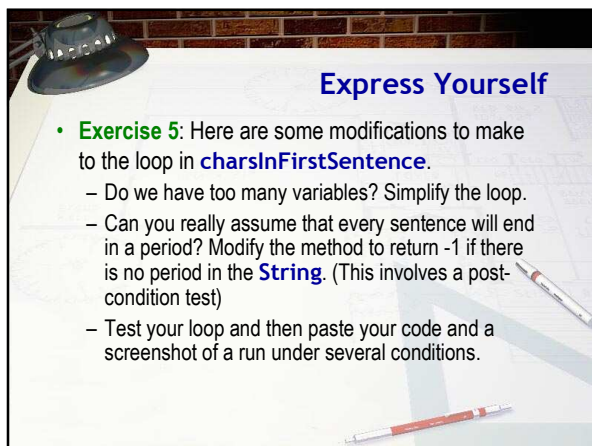


The Final Results

- **Step 6:** Make postcondition tests
- Ask yourself

Has the loop reached its goal?

- Loop exits because bound is reached, not because the goal is necessarily accomplished
- In our case (counter) no test is necessary
- If averaging, for instance, would need to make sure that the loop had actually been entered



Express Yourself

- **Exercise 5:** Here are some modifications to make to the loop in `charsInFirstSentence`.
- Do we have too many variables? Simplify the loop.
- Can you really assume that every sentence will end in a period? Modify the method to return -1 if there is no period in the **String**. (This involves a post-condition test)
- Test your loop and then paste your code and a screenshot of a run under several conditions.

Iterators

- Objects or methods that allow you to sequentially access all of the elements in a collection, regardless of how those elements are physically stored in memory
 - We'll look at some more sophisticated iterators later
 - For now, let's look at the **Scanner** as an iterator
- In addition to reading the keyboard, a **Scanner** can:
 - Read and process each line of text in a text file
 - Process each individual word or token in a line of text
- **Exercise 6:** Import **WordCounter.java** into **ic14**

Counting Words

- Locate the **countWords()** method, and then construct a **Scanner** object using the **text** variable

```
45: public int countWords()
46: {
47:     // Retrieve the string from the text area
48:     String text = wordsTA.getText();
49:     Scanner wordScan = new Scanner(text);
50: }
51:
```

- Note that the **Scanner** object will “step through” the **text String**, just like it steps through keyboard input
- The **Scanner** sees both as a stream of characters

Counting Words

- **Step 3:** All iterators have a **hasNext()** method
 - Use that as the bounds of our loop

```
53: int numWords = 0;
54: // Loop through the text and retrieve each word
55: while (wordScan.hasNext())
56: {
57:     // Loop through the text and retrieve each word
58:     while (wordScan.hasNext())
59:     {
60:         numWords++; // count the word
61:         wordScan.next(); // extract the word
62:     }
63: return numWords; // return the word
64:
```

Iterators and Text Files

- You can use a **Scanner** to read the lines in a text file
- Exercise 8:** Copy **moby.txt** into your **ic14** project folder. Then, add a new class named **LineCounter**.
 - Generate comments and a **main()** method
 - Add import **java.io.***; to the top of the file
 - Add **throws Exception** to **main()** signature
 - Construct a **Scanner** like this:

```
Scanner reader  
    = new Scanner(new File("moby.txt"));
```

Reading Text

- Use **hasNext()** and **nextLine()** to read text
 - Using **next()** would read next **word** instead

```
public static void main(String[] args) throws Exception  
{  
    Scanner reader = new Scanner(new File("moby.txt"));  
  
    int numLines = 0;  
    while (reader.hasNext())  
    {  
        reader.nextLine();  
        numLines++;  
    }  
  
    System.out.printf("%d lines in Moby Dick.", numLines);  
}
```

Card Demo

- Last two topics left to cover in Chapter 5 and 6 are **enumerated types** and the **Random** class.
- Let's return to our playing card example and create a class that represents playing cards
- Exercise 9:** Copy the cards folder and add it to your project. Then import the **CardDemo** program. Shoot a screenshot of your workspace.

CardDemo.java

- **CardDemo** creates a stack of **Card** objects on table
 - Some parts are missing. Let's start with **Card**.

```

31 add(table);
32
33 // Place suit where the deck of cards should go
34 Card c = new Card(Rank.TWO, Suit.CLUBS);
35 GPoint loc = new GPoint(c.getWidth()/2.0, c.getHeight()/2.0);
36
37
38 // Add all the cards to the deck
39 for (Suit suit: Suit.values())
40 {
41     for (Rank rank: Rank.values())
42     {
43         c = new Card(rank, suit);
44         add(c, loc);
45     }
46 }
47
48 // Listen for mouse events

```

The Card Class

- Create a new class **Card**, **acm.graphics.GImage**, no main method, comments

Modeling Playing Cards

- Every playing card has two defining data elements
 - **Rank**: 2-10, Jack, Queen, King, Ace
 - **Suit**: Clubs, Spades, Diamonds, Hearts
- We'll represent these as instance variables
 - The question is, what kind of data?
 - String? Assume we have a constructor like this:
 - **Card(String rank, String suit);**
 - User writes **Card c = new Card("Ac", "Clubs");**
- Compiler can't catch this, but **enumerated types** can

Enumerations

- Enumerations are user-defined types like classes
 - Each type has a restricted number of values
 - Think days of the week, months, compass points, etc.
- Create an enumerated type using the **enum** keyword
 - These types are **strongly type-checked**
 - A **better** way to create named constants
- Exercise 10:** In Eclipse, create two new Enums, one named **Rank** and one named **Suit**. Shoot a screenshot of your workspace.



Specifying Values

- Need a name for each value. Since they are constant, its convention to use upper-case for the name



Comma separated list of names.

Exercise 11: Add values for Rank. Start at TWO, end with ACE.

Using Enumerated Types

- Use them like an "integer" wrapper type

```

Suit suit;           // an enum variable
suit = Suit.HEARTS;  // initializing
suit == Suit.CLUBS;  // compare with != and ==
switch(suit) {       // use as a switch selector
    suit.ordinal()    // position in list
    Suit.values().length // elements in enum list
    Suit.values()[n]  // elements number n
    Card(Rank rank, Suit suit) // formal parameters
  
```

Constructing Cards

- Let's add our fields and a constructor to **Card**
 - Add a **private Rank** and **Suit** instance variable
 - Create a constructor that takes a **Rank** and **Suit**
 - Use **super("cards/b.gif");** as the first line
 - This initializes the **GImage** part using the card back
 - Assign both parameters to the fields you defined
- Everything now compiles except one missing method
 - Add a **private boolean** field, **isFaceDown**
 - Add a void method **flip()** that just toggles the field
- Exercise 12:** run and shoot a screen-shot.

Accessors and Mutators

- Eclipse will write a lot of code for you
 - Let's start with accessors for the our fields

Select only accessors

Place after constructor.

Flipping the Card

- To display the card image when we flip it, we need to generate the correct name, then use **setImage()**
 - Each card image is **"cards/" + rank + suit + ".gif"**
 - Ranks go from **2-9,t,q,k,a**
 - Suits go **c, s, d, h** (the order we declared them)
- We can do that several different ways
 - We can use two methods, generating the rank and suit symbols, using a **switch** statement
 - We can use **ordinal()** along with **substring()**
 - We can add methods to the **Rank** and **Suit** enums

Changing flip()

- Here's the change to the `flip()` method

```
/**
 * Flips the card over.
 */
public void flip()
{
    if (isFaceDown)
        setImage(getFileName());
    else
        setImage("cards/b.gif");
    isFaceDown = !isFaceDown;
}
```



Getting the File Name

- Here's the `getFileName()` method:

```
/**
 * Calculates the file-name for this card.
 */
private String getFileName()
{
    return "cards/" + rankSymbol() + suitSymbol() + ".gif";
}
```

The suitSymbol() Method

- Calculates the symbol based on its **ordinal** position
- **Real drawback**: breaks if you change order in enum

```
/**
 * Returns the suit symbol.
 */
private String suitSymbol()
{
    int pos = suit.ordinal();
    return "csdh".substring(pos, pos+1);
}
```

Position in enum definition



Only works if this is in-synch

The rankSymbol() Method

- Uses **switch**, so still works if values are reordered in enum definition
- Exercise 13:** Run, make sure you can drag and flip the cards. Snap a pic.

```
/**
 * Calculates the rank symbol using a switch.
 */
private String rankSymbol()
{
    switch(rank)
    {
        case TWO: return "2";
        case THREE: return "3";
        case FOUR: return "4";
        case FIVE: return "5";
        case SIX: return "6";
        case SEVEN: return "7";
        case EIGHT: return "8";
        case NINE: return "9";
        case TEN: return "c";
        case JACK: return "j";
        case QUEEN: return "q";
        case KING: return "k";
        default: return "a";
    }
}
```

The Random Class

- The **java.util.Random** class provides a more complete set of random number generating functions
 - Unlike **Math.random()**, these are not static
 - You first create a **Random** object like this:

```
Random gen1 = new Random();
Random gen2 = new Random(20L);
```

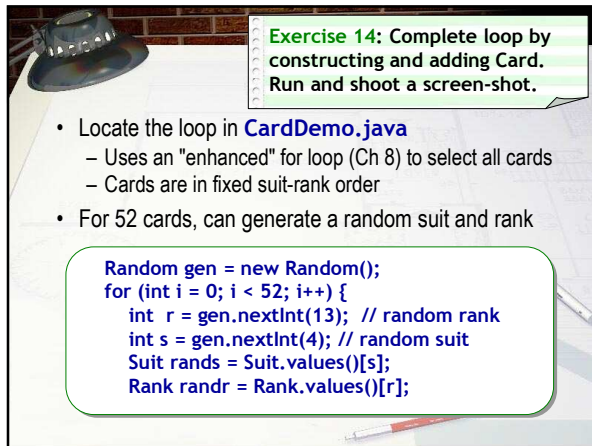
- The 20L passed to the second constructor is a **seed**
- Same sequence of numbers each time it is used
- Default constructor uses the system clock for the seed

Random Numbers

- Once you have a **Random** object, use it like this:

```
int iNum = r1.nextInt();
float fNum = r1.nextFloat();
double dNum = r1.nextDouble();
double gNum = r1.nextGaussian();
int iNum2 = r1.nextInt(5); // 0 - 4
```

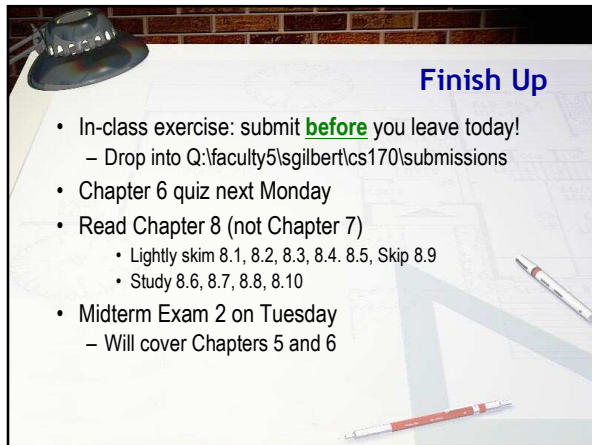
- Gaussian distribution mean at 0.0 and std dev of 1.0.
 - Both positive and negative numbers
 - Possible values > 1.0



Exercise 14: Complete loop by constructing and adding Card. Run and shoot a screen-shot.

- Locate the loop in **CardDemo.java**
 - Uses an "enhanced" for loop (Ch 8) to select all cards
 - Cards are in fixed suit-rank order
- For 52 cards, can generate a random suit and rank

```
Random gen = new Random();
for (int i = 0; i < 52; i++) {
    int r = gen.nextInt(13); // random rank
    int s = gen.nextInt(4); // random suit
    Suit rands = Suit.values()[s];
    Rank randr = Rank.values()[r];
```



Finish Up

- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Chapter 6 quiz next Monday
- Read Chapter 8 (not Chapter 7)
 - Lightly skim 8.1, 8.2, 8.3, 8.4, 8.5, Skip 8.9
 - Study 8.6, 8.7, 8.8, 8.10
- Midterm Exam 2 on Tuesday
 - Will cover Chapters 5 and 6
